

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 10 – API REST

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento especifica a arquitetura e o contrato da **API REST** do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é definir de forma padronizada como os clientes da aplicação (Frontend Web, aplicações móveis, integrações externas e futuros serviços) deverão consumir os recursos disponibilizados pelo backend.

A API foi projetada seguindo os princípios da arquitetura **REST (Representational State Transfer)**, adotando comunicação baseada em HTTP/HTTPS, troca de dados em formato JSON, autenticação por JWT e versionamento de endpoints.

Além de servir como contrato entre frontend e backend, este documento estabelece diretrizes para desenvolvimento, testes, integração contínua e documentação automática da API por meio do OpenAPI/Swagger.

1.2 Escopo

Este documento abrange:

- arquitetura da API REST;
- padrões de comunicação HTTP;
- autenticação e autorização;
- estrutura de requisições e respostas;
- códigos de status HTTP;
- tratamento de erros;
- versionamento da API;
- contratos dos endpoints;
- paginação, ordenação e filtros;
- upload e download de arquivos;
- monitoramento e observabilidade da API;
- documentação OpenAPI;
- diretrizes de evolução e compatibilidade.

Não fazem parte deste documento os detalhes internos de implementação do backend, descritos no **Documento 06 – Arquitetura Backend**, nem os aspectos de infraestrutura tratados no **Documento 09 – Docker**.

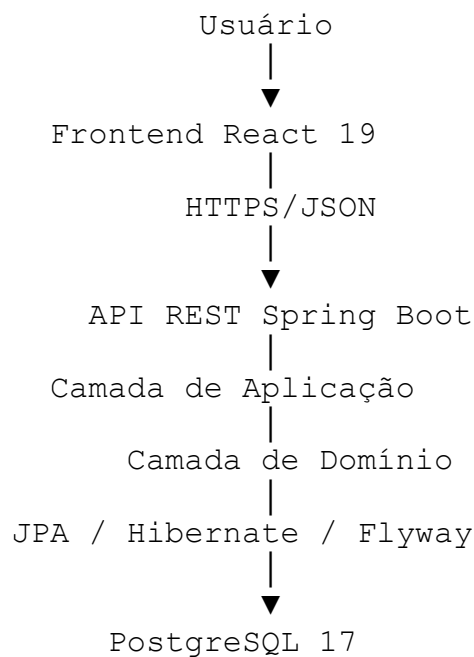
1.3 Objetivos da API

A API do GTP foi concebida para atender aos seguintes objetivos:

- disponibilizar serviços REST padronizados;
- permitir comunicação segura entre frontend e backend;
- possibilitar futuras integrações com sistemas externos;
- facilitar a evolução da plataforma por meio de versionamento;
- garantir consistência dos contratos entre clientes e servidor;
- reduzir acoplamento entre as camadas da aplicação;
- apoiar a automação de testes e documentação.

1.4 Visão Geral da Arquitetura

A API ocupa a camada intermediária entre a interface do usuário e os serviços de persistência.



Toda comunicação entre clientes e servidor ocorrerá exclusivamente por meio da API REST.

1.5 Características Gerais

A API possuirá as seguintes características:

- arquitetura RESTful;

- comunicação via HTTPS;
- troca de dados em JSON;
- autenticação utilizando JWT;
- autorização baseada em papéis (RBAC);
- operações idempotentes quando aplicável;
- suporte a paginação;
- suporte a filtros;
- suporte a ordenação;
- versionamento por URI;
- documentação OpenAPI;
- compatibilidade com Swagger UI.

1.6 Tecnologias Utilizadas

A implementação da API utilizará as seguintes tecnologias.

Tecnologia	Finalidade
Java 21 LTS	Linguagem principal da aplicação.
Spring Boot 3	Framework para construção da API.
Spring Web	Implementação dos endpoints REST.
Spring Security	Segurança da aplicação.
JWT	Autenticação baseada em tokens.
Spring Validation	Validação de dados de entrada.
Spring Data JPA	Persistência de dados.
Hibernate	Implementação ORM.
PostgreSQL 17	Banco de dados relacional.
Flyway	Versionamento do esquema do banco.
OpenAPI 3	Especificação da documentação da API.
Swagger UI	Interface interativa para exploração da API.

1.7 Consumidores da API

A API poderá ser consumida por diferentes tipos de clientes.

Cliente	Finalidade
Frontend React	Interface principal do sistema.
Aplicativos móveis (futuro)	Consumo das funcionalidades em dispositivos móveis.
Serviços internos	Comunicação entre módulos da plataforma.
Sistemas externos autorizados	Integrações futuras.
Ferramentas de testes	Validação automática dos contratos da API.

Todos os consumidores deverão obedecer aos contratos estabelecidos neste documento.

1.8 Princípios REST

A API seguirá os princípios fundamentais da arquitetura REST.

Separação Cliente-Servidor

A interface do usuário será completamente desacoplada da lógica de negócio.

Comunicação Stateless

Cada requisição deverá conter todas as informações necessárias para seu processamento.

O servidor não manterá estado de sessão entre as requisições.

Recursos

Todos os elementos da aplicação serão representados como recursos.

Exemplos:

- usuários;
- congregações;
- territórios;
- publicadores;
- designações;
- visitas;
- estudos bíblicos;
- relatórios.

Representações

Os recursos serão representados em formato JSON.

Interface Uniforme

Todos os endpoints seguirão convenções padronizadas para facilitar a utilização da API.

1.9 Convenções Gerais

Serão adotadas as seguintes convenções.

URLs

- letras minúsculas;
- palavras separadas por hífen quando necessário;
- nomes no plural para coleções.

Exemplos:

```
/api/v1/usuarios  
/api/v1/congregacoes  
/api/v1/territorios  
/api/v1/publicadores
```

JSON

As propriedades utilizarão **camelCase**.

Exemplo:

```
{  
  "nomeCompleto": "João Silva",  
  "dataNascimento": "1995-08-15",  
  "telefone": "(55) 99999-9999"  
}
```

Datas

Padrão ISO-8601.

Exemplo:

```
2026-07-10  
Data e hora:  
2026-07-10T14:30:00Z
```

Charset

Todas as respostas utilizarão UTF-8.

Content-Type

application/json

1.10 Versionamento

A API utilizará versionamento por URI.

Exemplo:

```
/api/v1/
Versões futuras:
/api/v2/
/api/v3/
```

Essa estratégia permite evolução da API sem interromper clientes existentes.

1.11 Compatibilidade

A evolução da API observará os seguintes princípios:

- preservar compatibilidade sempre que possível;
- evitar alterações incompatíveis na mesma versão;
- documentar mudanças relevantes;
- utilizar depreciação antes da remoção de funcionalidades;
- manter histórico de versões.

1.12 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral onde a API está inserida.
Documento 06 – Arquitetura Backend	Especifica a implementação da API no backend Spring Boot.

Documento	Relação
Documento 07 – Arquitetura Frontend	Descreve o consumo da API pela aplicação React.
Documento 08 – Segurança	Define autenticação, autorização e políticas de proteção aplicadas à API.
Documento 09 – Docker	Descreve a infraestrutura utilizada para implantação da API.

1.13 Conclusão do Capítulo

Este capítulo apresentou os fundamentos da API REST do GTP, definindo seus objetivos, escopo, princípios arquiteturais e convenções de desenvolvimento. A padronização aqui estabelecida servirá como base para todos os contratos de comunicação entre clientes e servidor, garantindo consistência, interoperabilidade e facilidade de evolução da plataforma.

2 – Arquitetura Geral da API REST

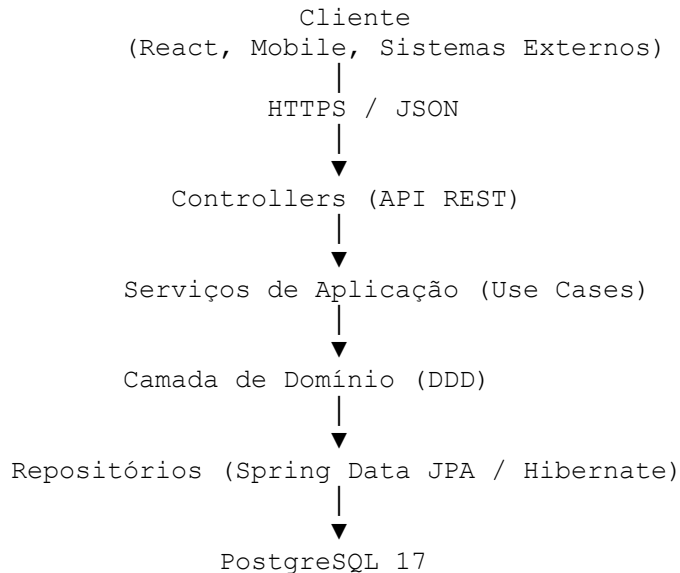
2.1 Objetivo

Este capítulo descreve a arquitetura geral da API REST do **Gestor de Territórios e Publicações (GTP)**, apresentando sua organização em camadas, o fluxo de processamento das requisições HTTP, a integração entre os componentes do backend e as diretrizes que garantem consistência, segurança e escalabilidade.

A arquitetura foi concebida para suportar o crescimento da aplicação, facilitar testes automatizados e permitir futuras integrações com outros sistemas.

2.2 Visão Geral da Arquitetura

A API atua como intermediária entre os clientes da aplicação e a camada de persistência de dados.



Cada camada possui responsabilidades bem definidas, reduzindo dependências e facilitando a evolução da plataforma.

2.3 Arquitetura em Camadas

A API será organizada nas seguintes camadas:

Camada	Responsabilidade
Apresentação	Receber requisições HTTP e retornar respostas JSON.
Aplicação	Orquestrar os casos de uso da aplicação.
Domínio	Implementar regras de negócio e entidades.
Persistência	Acessar o banco de dados utilizando JPA/Hibernate.
Infraestrutura	Segurança, logs, mensageria, integrações e configurações técnicas.

Cada camada poderá depender apenas da camada imediatamente inferior, respeitando o princípio da inversão de dependência.

2.4 Fluxo de uma Requisição

O processamento de uma requisição seguirá o fluxo abaixo:

```
Cliente
↓
Requisição HTTP
↓
Spring Security
↓
Controller
↓
Validação
↓
Serviço de Aplicação
↓
Domínio
↓
Repositório
↓
Banco de Dados
↓
Entidade
↓
DTO
↓
Resposta JSON
```

Esse fluxo garante separação clara entre responsabilidades e facilita o tratamento de erros e auditoria.

2.5 Controllers

Os Controllers representam a camada de entrada da API.

Responsabilidades:

- receber requisições HTTP;
- validar parâmetros básicos;
- delegar o processamento aos serviços;
- retornar respostas padronizadas;
- definir códigos HTTP apropriados.

Os Controllers não deverão conter regras de negócio.

Exemplo de responsabilidades:

- autenticação;
- gerenciamento de usuários;
- gestão de congregações;
- controle de territórios;

- designações;
- relatórios.

2.6 Serviços de Aplicação

A camada de aplicação implementará os casos de uso do sistema.

Principais responsabilidades:

- coordenar operações;
- aplicar regras de negócio de alto nível;
- controlar transações;
- integrar diferentes agregados do domínio;
- comunicar-se com serviços externos quando necessário.

Cada serviço representará uma funcionalidade específica da plataforma.

2.7 Camada de Domínio

O domínio concentra as regras de negócio do GTP.

Elementos principais:

- entidades;
- objetos de valor;
- agregados;
- serviços de domínio;
- eventos de domínio;
- interfaces de repositórios.

Essa camada deve permanecer independente de frameworks e tecnologias específicas.

2.8 Camada de Persistência

A persistência será implementada utilizando:

- Spring Data JPA;
- Hibernate;
- PostgreSQL 17.

Responsabilidades:

- consultas;
- gravações;
- atualizações;
- exclusões;
- paginação;
- filtros;
- ordenação.

Toda interação com o banco ocorrerá por meio de repositórios.

2.9 Data Transfer Objects (DTOs)

A comunicação entre a API e seus consumidores será realizada por meio de DTOs.

Objetivos:

- desacoplar entidades do domínio;
- controlar os dados expostos;
- facilitar versionamento;
- reduzir acoplamento entre backend e frontend.

Tipos de DTO:

Tipo	Finalidade
Request DTO	Dados recebidos da requisição.
Response DTO	Dados enviados ao cliente.
Summary DTO	Representações resumidas.
Detail DTO	Representações completas.

2.10 Serialização JSON

A API utilizará JSON como formato padrão de comunicação.

Diretrizes:

- propriedades em *camelCase*;
- datas no padrão ISO-8601;
- valores nulos omitidos quando apropriado;
- codificação UTF-8;
- estrutura consistente entre endpoints.

2.11 Validação de Entrada

Toda entrada recebida deverá ser validada antes da execução das regras de negócio.

Exemplos de validações:

- campos obrigatórios;
- tamanho mínimo e máximo;
- formatos de e-mail e telefone;
- datas válidas;
- unicidade de informações;
- consistência entre campos relacionados.

Erros de validação deverão resultar em respostas padronizadas com detalhes suficientes para correção pelo cliente.

2.12 Tratamento Global de Exceções

A API utilizará um mecanismo centralizado para tratamento de exceções.

Objetivos:

- evitar duplicação de código;
- padronizar mensagens de erro;
- registrar falhas em logs;
- retornar códigos HTTP apropriados.

Categorias de exceções:

- validação;
- autenticação;
- autorização;
- recurso não encontrado;
- conflito de dados;
- regras de negócio;
- erros internos.

2.13 Integração com Spring Security

Toda requisição protegida passará pelo fluxo de segurança.

Etapas:

1. recebimento da requisição;
2. validação do token JWT;

3. identificação do usuário;
4. carregamento das permissões;
5. autorização para execução do recurso;
6. encaminhamento ao Controller.

Requisições não autorizadas serão interrompidas antes de alcançar a lógica de negócio.

2.14 Integração com JPA e Hibernate

A persistência seguirá o fluxo:

```
Controller
↓
Service
↓
Repository
↓
Hibernate
↓
PostgreSQL
```

O uso do ORM reduz código repetitivo e facilita a manutenção da camada de dados.

2.15 Transações

As operações que alteram o estado da aplicação deverão ser executadas dentro de transações.

Objetivos:

- garantir consistência;
- preservar integridade dos dados;
- evitar estados intermediários;
- permitir rollback em caso de falhas.

2.16 Observabilidade

Cada requisição deverá gerar informações úteis para monitoramento.

Serão registrados:

- horário da requisição;
- usuário autenticado;

- endpoint acessado;
- tempo de processamento;
- código HTTP retornado;
- exceções, quando existirem.

Esses registros apoiarão auditorias e diagnósticos.

2.17 Princípios Arquiteturais

A implementação da API seguirá os seguintes princípios:

- responsabilidade única;
- baixo acoplamento;
- alta coesão;
- inversão de dependência;
- separação de responsabilidades;
- reutilização de componentes;
- orientação a contratos;
- facilidade de testes.

2.18 Boas Práticas

Durante o desenvolvimento da API deverão ser observadas as seguintes recomendações:

- manter Controllers enxutos;
- concentrar regras de negócio na camada de domínio e aplicação;
- utilizar DTOs para comunicação externa;
- evitar exposição direta das entidades;
- documentar todos os endpoints;
- utilizar códigos HTTP corretos;
- padronizar respostas;
- registrar eventos relevantes em logs.

2.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral em que a API está inserida.
Documento 06 – Arquitetura Backend	Detalha a implementação das camadas descritas neste capítulo.
Documento 07 – Arquitetura Frontend	Especifica como o frontend consome os serviços da API.

Documento	Relação
Documento 08 – Segurança	Define autenticação, autorização e políticas aplicadas ao fluxo das requisições.
Documento 09 – Docker	Descreve a infraestrutura responsável pela execução da API.

2.20 Conclusão do Capítulo

A arquitetura geral da API REST do GTP estabelece uma base sólida para o desenvolvimento de serviços escaláveis, seguros e de fácil manutenção. A organização em camadas, a separação clara de responsabilidades, o uso de DTOs, o tratamento centralizado de exceções, a integração com Spring Security e JPA/Hibernate e a adoção de boas práticas arquiteturais garantem uma API consistente, preparada para evoluções futuras e alinhada aos padrões modernos de desenvolvimento de software.

3 – Padrões REST, Recursos e Convenções da API

3.1 Objetivo

Este capítulo estabelece os padrões arquiteturais e as convenções que deverão ser seguidos por todos os recursos e endpoints da API REST do GTP.

A padronização visa:

- facilitar o consumo da API;
- reduzir ambiguidades;
- garantir consistência entre módulos;
- simplificar a manutenção;
- favorecer a evolução da plataforma.

Todas as novas funcionalidades deverão obedecer às diretrizes aqui definidas.

3.2 Princípios REST Aplicados ao GTP

A API do GTP seguirá os princípios da arquitetura REST.

Cliente-Servidor

A interface do usuário permanecerá desacoplada da lógica de negócio.

Stateless

Cada requisição deverá conter todas as informações necessárias para seu processamento.

O servidor não armazenará estado de sessão entre chamadas.

Interface Uniforme

Todos os recursos seguirão convenções comuns de nomenclatura, métodos HTTP e estrutura de mensagens.

Recursos Identificáveis

Cada recurso possuirá uma URI única.

Exemplo:

```
/api/v1/publicadores/125
```

Representações

Os recursos serão representados em JSON.

3.3 Organização dos Recursos

Os recursos representarão entidades de negócio.

Exemplos:

Recurso	Endpoint Base
Usuários	/api/v1/usuarios
Congregações	/api/v1/congregacoes
Territórios	/api/v1/territorios
Quadras	/api/v1/quadras
Endereços	/api/v1/enderecos
Publicadores	/api/v1/publicadores
Designações	/api/v1/designacoes
Visitas	/api/v1/visitas
Estudos Bíblicos	/api/v1/estudos-biblicos
Relatórios	/api/v1/relatorios

Os nomes dos recursos deverão refletir conceitos do domínio da aplicação.

3.4 Convenções de URLs

As URLs deverão obedecer às seguintes regras:

- utilizar letras minúsculas;
- evitar caracteres especiais;
- utilizar hífen (-) quando necessário;
- utilizar substantivos no plural;
- evitar verbos na URI;
- manter hierarquia lógica entre recursos.

Exemplos corretos:

`/api/v1/publicadores`

`/api/v1/territorios`

`/api/v1/estudos-biblicos`

Exemplos a evitar:

`/api/v1/listarPublicadores`

`/api/v1/getUsuarios`

`/api/v1/criarTerritorio`

A ação será definida pelo método HTTP, e não pelo nome da URL.

3.5 Métodos HTTP

A API utilizará os métodos HTTP conforme suas responsabilidades.

Método Finalidade		Idempotente
GET	Consultar recursos	Sim
POST	Criar novos recursos	Não
PUT	Substituir integralmente um recurso	Sim
PATCH	Atualizar parcialmente um recurso	Não (depende da implementação)
DELETE	Remover um recurso	Sim

GET

Utilizado para consultas.

Exemplo:

```
GET /api/v1/publicadores
```

POST

Utilizado para criação.

```
POST /api/v1/publicadores
```

PUT

Atualização completa.

```
PUT /api/v1/publicadores/35
```

PATCH

Atualização parcial.

```
PATCH /api/v1/publicadores/35
```

DELETE

Remoção lógica ou física conforme regra de negócio.

```
DELETE /api/v1/publicadores/35
```

3.6 Identificação dos Recursos

Cada recurso possuirá um identificador único.

Exemplo:

```
/api/v1/publicadores/250
```

O identificador será imutável durante todo o ciclo de vida do recurso.

3.7 Recursos Aninhados

Quando houver relação hierárquica, poderão ser utilizados recursos aninhados.

Exemplo:

```
/api/v1/congregacoes/3/publicadores
```

Outro exemplo:

```
/api/v1/territorios/18/designacoes
```

Esse padrão melhora a expressividade da API.

3.8 Parâmetros de Consulta (Query Parameters)

Os parâmetros de consulta serão utilizados para operações como:

- filtros;
- paginação;
- ordenação;
- pesquisa.

Exemplo:

```
GET /api/v1/publicadores?nome=Carlos
```

Exemplo com múltiplos parâmetros:

```
GET /api/v1/publicadores?ativo=true&pagina=0&tamanho=20
```

3.9 Paginação

Listagens deverão utilizar paginação.

Parâmetros recomendados:

Parâmetro Finalidade

pagina Número da página.

tamanho Quantidade de registros.

ordenar Campo de ordenação.

direcao ASC ou DESC.

Exemplo:

```
GET  
/api/v1/publicadores?pagina=1&tamanho=25&ordenar=nome&direcao=ASC
```

3.10 Ordenação

A ordenação poderá ser aplicada a qualquer coleção.

Exemplos:

```
?ordenar=nome
```

```
?ordenar=dataCadastro
```

```
?ordenar=territorio
```

A direção poderá ser:

```
ASC
```

```
DESC
```

3.11 Filtros

Os recursos poderão ser filtrados por diferentes critérios.

Exemplos:

```
?ativo=true
```

```
?bairro=Centro
```

```
?status=DISPONIVEL
```

```
?dataInicial=2026-01-01&dataFinal=2026-12-31
```

Filtros poderão ser combinados conforme necessidade.

3.12 Pesquisa

Pesquisas textuais deverão utilizar parâmetros específicos.

Exemplo:

```
GET /api/v1/publicadores?pesquisa=joao
```

A implementação poderá considerar:

- nome;
- telefone;
- e-mail;
- código do publicador.

3.13 Operações em Lote

Quando necessário, a API poderá suportar operações em lote.

Exemplo:

```
POST /api/v1/publicadores/lote
```

Casos de uso:

- importação;
- atualização em massa;

- exclusão lógica em grupo.

3.14 Idempotência

Operações deverão respeitar os princípios da idempotência.

Método Comportamento

GET Sempre idempotente.

PUT Deve produzir o mesmo resultado em chamadas repetidas.

DELETE Chamadas repetidas não devem causar efeitos adicionais.

POST Pode gerar novos recursos a cada chamada.

3.15 Versionamento

Todos os recursos deverão conter a versão da API.

Exemplo:

```
/api/v1/
```

Versões futuras:

```
/api/v2/
```

Mudanças incompatíveis deverão ocorrer apenas em novas versões.

3.16 Convenções de Nomenclatura

Os nomes dos recursos deverão utilizar:

- português;
- substantivos;
- plural;
- linguagem consistente com o domínio.

Exemplos:

- usuários;
- congregações;
- territórios;
- publicadores;
- designações.

Evitar abreviações e siglas não documentadas.

3.17 Compatibilidade Evolutiva

A evolução da API deverá observar:

- manutenção dos contratos existentes;
- depreciação gradual de endpoints;
- documentação das alterações;
- coexistência temporária entre versões.

Essas práticas reduzem impactos para clientes já integrados.

3.18 Boas Práticas

Durante o desenvolvimento dos endpoints deverão ser observadas as seguintes recomendações:

- utilizar corretamente os métodos HTTP;
- evitar verbos nas URLs;
- manter nomes consistentes;
- utilizar recursos aninhados apenas quando agregarem clareza;
- implementar paginação em todas as listagens;
- padronizar filtros e ordenação;
- preservar compatibilidade entre versões;
- documentar todos os recursos no OpenAPI.

3.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a estrutura geral onde a API está inserida.
Documento 06 – Arquitetura Backend	Especifica a implementação dos Controllers e serviços que seguem estes padrões.
Documento 07 – Arquitetura Frontend	Consome os recursos definidos neste capítulo.

Documento	Relação
Documento 08 – Segurança	Define as políticas de autenticação e autorização aplicadas aos recursos.
Documento 09 – Docker	Descreve a infraestrutura responsável pela execução da API.

3.20 Conclusão do Capítulo

Os padrões REST e as convenções definidos neste capítulo garantem uniformidade na construção da API do GTP. A utilização consistente de métodos HTTP, URLs padronizadas, recursos bem modelados, paginação, filtros, ordenação e versionamento facilita o consumo da API por diferentes clientes, melhora a experiência dos desenvolvedores e contribui para a manutenção e evolução do sistema.

4 – Autenticação, Autorização e Segurança da API

4.1 Objetivo

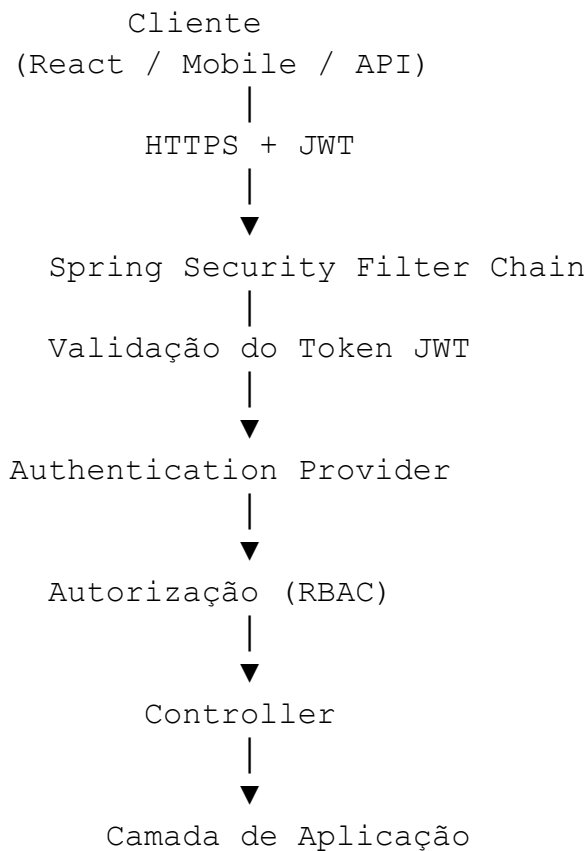
Este capítulo define os mecanismos de autenticação, autorização e proteção da API REST do GTP.

Seu objetivo é garantir que apenas usuários autenticados e devidamente autorizados possam acessar os recursos da aplicação, protegendo informações sensíveis e assegurando a integridade das operações realizadas.

A arquitetura de segurança foi projetada para ser escalável, desacoplada e preparada para futuras integrações com provedores de identidade externos.

4.2 Arquitetura de Segurança

A segurança da API será baseada em uma arquitetura composta pelos seguintes componentes:



Todo acesso aos recursos protegidos deverá passar por esse fluxo.

4.3 Princípios de Segurança

A API seguirá os seguintes princípios:

- autenticação obrigatória para recursos protegidos;
- autorização baseada em papéis e permissões;
- comunicação exclusivamente por HTTPS;
- ausência de estado de sessão (*stateless*);
- princípio do menor privilégio (*Least Privilege*);
- defesa em profundidade (*Defense in Depth*);
- rastreabilidade de ações por meio de auditoria.

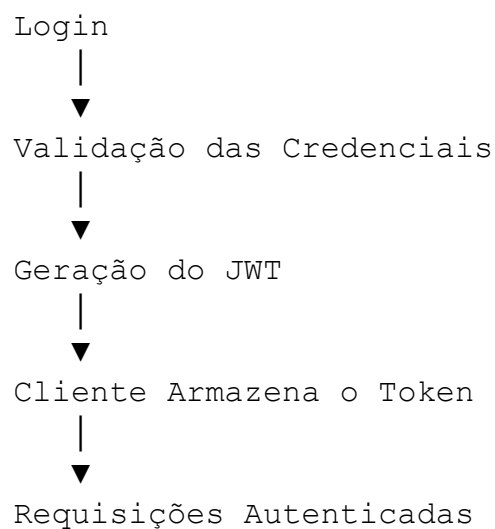
4.4 Processo de Autenticação

A autenticação será realizada por meio de credenciais válidas fornecidas pelo usuário.

Fluxo:

1. O cliente envia usuário e senha.
2. O backend valida as credenciais.
3. Em caso de sucesso, é emitido um **Access Token (JWT)**.
4. Opcionalmente, é emitido um **Refresh Token**.
5. O cliente utiliza o Access Token nas requisições subsequentes.

Fluxo resumido:



4.5 JSON Web Token (JWT)

O JWT será utilizado para autenticação sem estado (*stateless*).

O token conterá informações como:

- identificador do usuário;
- nome de usuário;
- papéis (*roles*);
- permissões;
- data de emissão;
- data de expiração.

As informações serão assinadas digitalmente para impedir alterações não autorizadas.

4.6 Estrutura do JWT

O token será composto por três partes:

HEADER.PAYLOAD.SIGNATURE

- **Header:** algoritmo e tipo do token.
- **Payload:** informações (claims).
- **Signature:** assinatura criptográfica.

O backend validará a assinatura em todas as requisições autenticadas.

4.7 Access Token

O Access Token será utilizado para autorizar o acesso aos recursos da API.

Características:

- curta duração;
- utilizado em todas as requisições protegidas;
- transmitido no cabeçalho HTTP Authorization;
- revogado em caso de comprometimento ou logout.

Exemplo:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

4.8 Refresh Token

O Refresh Token permitirá a emissão de um novo Access Token sem necessidade de novo login.

Benefícios:

- melhora a experiência do usuário;
- reduz autenticações frequentes;
- mantém o Access Token com tempo de vida reduzido.

O Refresh Token deverá possuir regras específicas de expiração e revogação.

4.9 Logout

Ao realizar logout:

- o cliente removerá os tokens armazenados;

- o backend poderá invalidar Refresh Tokens ativos;
- novas requisições utilizando tokens revogados deverão ser rejeitadas.

4.10 Autorização (RBAC)

O controle de acesso utilizará o modelo **Role-Based Access Control (RBAC)**.

Papéis previstos:

Papel	Responsabilidade
ADMINISTRADOR	Administração completa da plataforma.
SUPERINTENDENTE_SERVICO	Gestão da congregação e dos territórios.
ANCIÃO	Operações administrativas autorizadas.
SERVO_MINISTERIAL	Atividades operacionais específicas.
PUBLICADOR	Consulta e atualização dos próprios dados conforme permissões.

Novos papéis poderão ser adicionados conforme a evolução do sistema.

4.11 Permissões

Além dos papéis, a API poderá utilizar permissões granulares.

Exemplos:

- USUARIO_CRIAR
- USUARIO_EDITAR
- USUARIO_EXCLUIR
- TERRITORIO_VISUALIZAR
- TERRITORIO_DESIGNAR
- RELATORIO_EXPORTAR
- CONFIGURACAO_ALTERAR

Esse modelo permite maior flexibilidade na definição de acessos.

4.12 Proteção dos Endpoints

Os endpoints serão classificados em:

Tipo	Exemplo
Públicos	Login, renovação de token, documentação pública (quando habilitada).
Autenticados	Recursos acessíveis a qualquer usuário autenticado.
Restritos	Recursos disponíveis apenas para papéis específicos.

A política de acesso será definida de forma centralizada no Spring Security.

4.13 Controle de Acesso por Método

A autorização poderá ser aplicada diretamente nos métodos da camada de aplicação.

Exemplos de verificações:

- papel do usuário;
- permissões específicas;
- regras de negócio adicionais;
- vínculo com a congregação;
- proprietário do recurso.

Essa abordagem complementa a proteção realizada nos endpoints.

4.14 Expiração de Tokens

Os tokens deverão possuir tempo de validade limitado.

Recomendações:

Token	Tempo sugerido
Access Token	15 a 30 minutos
Refresh Token	7 a 30 dias

Os tempos poderão ser configurados conforme o ambiente de execução.

4.15 Revogação de Tokens

A API deverá permitir revogar tokens em situações como:

- logout;

- redefinição de senha;
- alteração de privilégios;
- suspeita de comprometimento;
- desativação do usuário.

A revogação reduz o risco de utilização indevida de credenciais.

4.16 Proteção contra Ataques

A API adotará medidas para mitigar ataques comuns, incluindo:

- força bruta;
- reutilização de tokens;
- interceptação de requisições;
- elevação de privilégios;
- injeção de comandos;
- exposição excessiva de dados.

Essas medidas complementam as diretrizes definidas no Documento 08 – Segurança.

4.17 Auditoria

Eventos de segurança deverão ser registrados.

Exemplos:

- login bem-sucedido;
- falhas de autenticação;
- logout;
- alteração de permissões;
- acesso a recursos críticos;
- tentativas de acesso negadas.

Os registros deverão conter data, hora, usuário e origem da requisição, respeitando as políticas de privacidade.

4.18 Boas Práticas

Durante a implementação da segurança da API deverão ser observadas as seguintes recomendações:

- utilizar HTTPS em todos os ambientes acessíveis externamente;
- proteger todos os endpoints sensíveis;
- validar JWT em cada requisição;
- limitar o tempo de vida dos tokens;
- aplicar o princípio do menor privilégio;
- registrar eventos de segurança;
- revisar periodicamente papéis e permissões;
- manter dependências de segurança atualizadas;
- seguir as recomendações da OWASP API Security Top 10.

4.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral onde a API está inserida.
Documento 06 – Arquitetura Backend	Descreve a implementação da autenticação e autorização no Spring Boot.
Documento 07 – Arquitetura Frontend	Define o armazenamento e o envio seguro dos tokens pelo cliente React.
Documento 08 – Segurança	Estabelece as políticas gerais de segurança, criptografia e auditoria utilizadas pela API.
Documento 09 – Docker	Define a infraestrutura segura para execução e implantação da API.

4.20 Conclusão do Capítulo

A arquitetura de autenticação, autorização e segurança da API do GTP garante que o acesso aos recursos seja realizado de forma controlada, segura e auditável. A utilização de JWT, RBAC, permissões granulares, comunicação via HTTPS, políticas de expiração e revogação de tokens, além da integração com o Spring Security, fornece uma base robusta para proteger a plataforma contra acessos indevidos e apoiar sua evolução futura.

5 – Estrutura das Requisições e Respostas da API

5.1 Objetivo

Este capítulo define a estrutura padronizada das requisições e respostas da API REST do GTP.

Seu objetivo é estabelecer um contrato único para comunicação entre clientes e servidor, assegurando consistência na troca de informações, facilidade de integração e redução de ambiguidades durante o desenvolvimento.

Todas as interfaces da API deverão obedecer aos padrões apresentados neste documento.

5.2 Estrutura Geral das Requisições

Toda requisição HTTP será composta pelos seguintes elementos:

- método HTTP;
- URL do recurso;
- cabeçalhos (*Headers*);
- parâmetros de rota (*Path Parameters*);
- parâmetros de consulta (*Query Parameters*);
- corpo da requisição (*Request Body*), quando aplicável.

Estrutura geral:

```
POST /api/v1/publicadores
Headers
Authorization
Content-Type
Body
JSON
```

5.3 Métodos HTTP

Os métodos suportados serão:

Método Utilização

GET Consulta de recursos.

POST Criação de novos recursos.

Método Utilização

PUT Atualização completa.

PATCH Atualização parcial.

DELETE Remoção lógica ou física.

Cada método deverá respeitar os princípios REST descritos no Capítulo 3.

5.4 Cabeçalhos HTTP (Headers)

Os cabeçalhos padronizados incluem:

Cabeçalho	Obrigatório	Finalidade
Authorization	Sim (quando autenticado)	Envio do JWT.
Content-Type	Sim	Tipo do conteúdo enviado.
Accept	Recomendado	Tipo de resposta esperado.
Accept-Language	Opcional	Idioma preferencial da resposta.
X-Correlation-Id	Opcional	Correlação entre requisições para rastreamento.

Exemplo:

```
Authorization: Bearer <jwt>
Content-Type: application/json
Accept: application/json
```

5.5 Path Parameters

Os parâmetros de rota identificam recursos específicos.

Exemplo:

```
GET /api/v1/publicadores/25
```

Neste exemplo:

```
id = 25
```

Outro exemplo:

```
GET /api/v1/congregacoes/8/territorios
```

5.6 Query Parameters

Utilizados para filtros, paginação, ordenação e pesquisa.

Exemplo:

```
GET /api/v1/publicadores?ativo=true
```

Exemplo completo:

```
GET  
/api/v1/publicadores?pagina=0&tamanho=20&ordenar=nome&  
direcao=ASC
```

Parâmetros recomendados:

Parâmetro Finalidade

pagina	Número da página.
tamanho	Quantidade de registros.
ordenar	Campo utilizado na ordenação.
direcao	ASC ou DESC.
pesquisa	Pesquisa textual.
ativo	Filtrar registros ativos.

5.7 Corpo da Requisição (Request Body)

Operações de criação e atualização utilizarão JSON.

Exemplo:

```
{  
  "nomeCompleto": "Carlos Alberto Souza",  
  "telefone": "(55)99999-1111",  
  "email": "carlos@email.com",  
  "ativo": true  
}
```

Todos os objetos enviados deverão respeitar as regras de validação definidas pelo backend.

5.8 Estrutura Geral das Respostas

As respostas seguirão um padrão único.

Estrutura:

```
{
  "success": true,
  "timestamp": "2026-07-10T15:30:00Z",
  "data": {},
  "message": "Operação realizada com sucesso."
}
```

Campos:

Campo	Descrição
--------------	------------------

success	Indica sucesso ou falha.
---------	--------------------------

timestamp	Data e hora da resposta.
-----------	--------------------------

data	Dados retornados.
------	-------------------

message	Mensagem descritiva.
---------	----------------------

5.9 Respostas de Consulta

Exemplo:

```
{
  "success": true,
  "timestamp": "2026-07-10T15:30:00Z",
  "data": {
    "id": 15,
    "nomeCompleto": "Carlos Alberto Souza",
    "telefone": "(55) 99999-1111"
  }
}
```

5.10 Respostas de Criação

Exemplo:

```
{
  "success": true,
  "message": "Publicador cadastrado com sucesso.",
  "data": {
    "id": 125
  }
}
```

Código HTTP esperado:

201 Created

5.11 Respostas de Atualização

Exemplo:

```
{
  "success": true,
  "message": "Registro atualizado com sucesso."
}
```

Código HTTP:

200 OK

ou

204 No Content

quando não houver corpo de resposta.

5.12 Respostas de Exclusão

Exemplo:

```
{
  "success": true,
  "message": "Registro removido com sucesso."
}
```

Código HTTP recomendado:

204 No Content

quando apropriado.

5.13 Paginação

As listagens utilizarão paginação padronizada.

Exemplo:

```
{
  "success": true,
  "data": [
    {}
  ],
  "page": {
    "numero": 0,
    "tamanho": 20,
    "totalElementos": 250,
    "totalPaginas": 13
  }
}
```

5.14 Ordenação

A resposta deverá refletir a ordenação solicitada.

Exemplo:

```
?ordenar=nome&direcao=ASC
```

ou

```
?ordenar=dataCadastro&direcao=DESC
```

5.15 Filtros

Filtros poderão ser combinados.

Exemplo:

```
?ativo=true
&bairro=Centro
&pesquisa=João
```

O backend deverá aplicar todos os filtros informados.

5.16 Estrutura de Erros

Todas as respostas de erro seguirão um padrão único.

Exemplo:

```
{
  "success": false,
  "timestamp": "2026-07-10T15:30:00Z",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "Existem erros de validação.",
  "details": []
}
```

5.17 Erros de Validação

Exemplo:

```
{
  "success": false,
  "status": 400,
  "error": "VALIDATION_ERROR",
  "details": [
    {
      "field": "nomeCompleto",
      "message": "Campo obrigatório."
    }
  ]
}
```

Esse formato facilita a exibição das mensagens no frontend.

5.18 Metadados

Algumas respostas poderão conter metadados adicionais.

Exemplo:

```
{
  "meta": {
    "versaoApi": "v1",
    "tempoProcessamento": 42
  }
}
```

Essas informações auxiliam monitoramento e diagnóstico.

5.19 Formato de Datas

Todas as datas utilizarão ISO-8601.

Exemplo:

```
2026-07-10
```

Data e hora:

```
2026-07-10T15:30:00Z
```

5.20 Codificação

A API utilizará:

```
UTF-8
```

Tipo de conteúdo:

```
application/json
```

5.21 Compressão

Em ambientes de produção poderá ser utilizada compressão HTTP (GZIP ou Brotli) para reduzir o tamanho das respostas e melhorar o desempenho da comunicação.

5.22 Idempotência

As respostas deverão respeitar os princípios definidos para cada método HTTP.

Operações repetidas de:

- GET;
- PUT;
- DELETE;

não deverão produzir efeitos inesperados.

5.23 Boas Práticas

Todas as respostas da API deverão:

- manter estrutura consistente;
- utilizar mensagens claras;
- retornar códigos HTTP apropriados;
- evitar exposição de informações sensíveis;
- documentar exemplos no OpenAPI;
- fornecer detalhes suficientes para tratamento de erros pelo cliente.

5.24 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura em que a API está inserida.
Documento 06 – Arquitetura Backend	Implementa os DTOs, Controllers e Responses definidos neste capítulo.
Documento 07 – Arquitetura Frontend	Consome as estruturas padronizadas de requisições e respostas.
Documento 08 – Segurança	Define os cabeçalhos de autenticação e políticas de proteção aplicadas às requisições.
Documento 09 – Docker	Disponibiliza a infraestrutura responsável pela execução da API.

5.25 Conclusão do Capítulo

A padronização das requisições e respostas da API do GTP estabelece um contrato consistente para comunicação entre clientes e servidor. A definição uniforme de cabeçalhos, parâmetros, estruturas JSON, paginação, tratamento de erros e metadados simplifica o desenvolvimento, melhora a experiência dos consumidores da API e facilita a manutenção, a documentação e a evolução da plataforma.

6 – Códigos HTTP, Tratamento Global de Erros e Padronização de Exceções

6.1 Objetivo

Este capítulo estabelece os padrões para utilização dos códigos de status HTTP e para o tratamento centralizado de exceções da API REST do GTP.

Os objetivos são:

- padronizar todas as respostas de erro;
- facilitar o consumo da API;
- reduzir ambiguidades na comunicação;
- melhorar a experiência dos desenvolvedores;
- permitir integração consistente entre frontend, backend e sistemas externos.

6.2 Princípios Gerais

A API deverá seguir os seguintes princípios:

- utilizar códigos HTTP apropriados;
- retornar mensagens claras e objetivas;
- evitar exposição de informações sensíveis;
- manter estrutura uniforme para erros;
- registrar exceções em logs;
- diferenciar erros técnicos de erros de negócio.

6.3 Estrutura Padronizada de Erros

Todas as respostas de erro seguirão uma estrutura baseada na RFC 7807, adaptada às necessidades do GTP.

Exemplo:

```
{
  "timestamp": "2026-07-10T18:20:35Z",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "title": "Erro de validação",
  "detail": "Um ou mais campos informados são inválidos.",
  "instance": "/api/v1/publicadores",
  "traceId": "6f3fdb9d-5f5d-4d17-a6d0-2b1d5a1c6d5a",
  "errors": [
    {
      "field": "nomeCompleto",
      "message": "O campo é obrigatório."
    }
  ]
}
```

6.4 Campos da Resposta de Erro

Campo	Descrição
-------	-----------

timestamp	Data e hora do erro (UTC).
-----------	----------------------------

status	Código HTTP retornado.
--------	------------------------

error	Código interno padronizado da aplicação.
-------	--

title	Título resumido do problema.
-------	------------------------------

detail	Descrição detalhada.
--------	----------------------

instance	Endpoint que originou o erro.
----------	-------------------------------

traceId	Identificador para rastreamento.
---------	----------------------------------

errors	Lista de erros de validação, quando aplicável.
--------	--

6.5 Códigos HTTP Utilizados

2xx – Sucesso

Código	Utilização
200 OK	Consulta ou atualização realizada com sucesso.
201 Created	Recurso criado com sucesso.
202 Accepted	Requisição aceita para processamento assíncrono.
204 No Content	Operação concluída sem conteúdo de retorno.

4xx – Erros do Cliente

Código	Utilização
400 Bad Request	Requisição inválida ou erro de validação.
401 Unauthorized	Usuário não autenticado ou token inválido.
403 Forbidden	Usuário autenticado sem permissão.
404 Not Found	Recurso não encontrado.
405 Method Not Allowed	Método HTTP não permitido.
409 Conflict	Conflito de dados (duplicidade, estado inválido etc.).
415 Unsupported Media Type	Tipo de conteúdo não suportado.
422 Unprocessable Entity	Dados válidos sintaticamente, mas inválidos pelas regras de negócio.
429 Too Many Requests	Limite de requisições excedido.

5xx – Erros do Servidor

Código	Utilização
500 Internal Server Error	Erro interno inesperado.
502 Bad Gateway	Falha em serviço intermediário.
503 Service Unavailable	Serviço temporariamente indisponível.
504 Gateway Timeout	Tempo limite excedido em integração externa.

6.6 Tratamento Global de Exceções

A API utilizará um mecanismo centralizado de tratamento de exceções (por exemplo, com `@RestControllerAdvice` no Spring Boot).

Responsabilidades:

- capturar exceções não tratadas;
- converter exceções em respostas HTTP padronizadas;
- registrar logs;
- impedir vazamento de detalhes internos da aplicação.

6.7 Erros de Validação

Erros gerados pela validação de entrada deverão retornar todos os campos inválidos.

Exemplo:

```
{
  "status": 400,
  "error": "VALIDATION_ERROR",
  "title": "Erro de validação",
  "errors": [
    {
      "field": "email",
      "message": "Formato de e-mail inválido."
    },
    {
      "field": "telefone",
      "message": "O telefone é obrigatório."
    }
  ]
}
```

Isso permite que o frontend apresente todas as inconsistências ao usuário em uma única resposta.

6.8 Erros de Autenticação

Quando o usuário não estiver autenticado ou o token for inválido:

Código HTTP:

401 Unauthorized

Exemplo:

```
{
  "status": 401,
  "error": "UNAUTHORIZED",
  "title": "Autenticação necessária",
  "detail": "O token informado é inválido ou expirou."
}
```

6.9 Erros de Autorização

Quando o usuário estiver autenticado, mas não possuir permissão:

Código HTTP:

403 Forbidden

Exemplo:

```
{
  "status": 403,
  "error": "ACCESS_DENIED",
  "title": "Acesso negado",
  "detail": "Você não possui permissão para executar esta operação."
}
```

6.10 Recursos Não Encontrados

Código HTTP:

404 Not Found

Exemplo:

```
{
  "status": 404,
  "error": "RESOURCE_NOT_FOUND",
  "title": "Recurso não encontrado",
  "detail": "O território solicitado não foi localizado."
}
```

6.11 Conflitos de Dados

Situações típicas:

- e-mail já cadastrado;
- nome duplicado;
- tentativa de cadastrar um recurso que viola restrições de unicidade.

Código HTTP:

```
409 Conflict
```

6.12 Violação de Regras de Negócio

Algumas requisições são válidas do ponto de vista técnico, mas violam regras do domínio.

Exemplos:

- designar um território já designado;
- encerrar uma visita inexistente;
- excluir uma congregação que ainda possui territórios vinculados.

Código HTTP recomendado:

```
422 Unprocessable Entity
```

6.13 Erros Internos

Erros inesperados deverão retornar uma resposta genérica.

Exemplo:

```
{
  "status": 500,
  "error": "INTERNAL_SERVER_ERROR",
  "title": "Erro interno",
  "detail": "Ocorreu um erro inesperado. Tente novamente mais tarde."
}
```

Detalhes técnicos deverão ser registrados apenas nos logs.

6.14 Rastreabilidade

Cada erro deverá possuir um identificador único (traceld) para facilitar:

- auditoria;
- correlação de logs;
- suporte técnico;
- diagnóstico de incidentes.

Esse identificador deverá ser propagado entre os serviços quando houver integrações.

6.15 Registro em Logs

Todas as exceções relevantes deverão ser registradas com informações como:

- data e hora;
- usuário autenticado (quando houver);
- endpoint acessado;
- método HTTP;
- endereço IP (observadas as políticas de privacidade);
- código HTTP;
- tipo da exceção;
- traceld.

6.16 Integração com Observabilidade

Os erros deverão ser integrados às ferramentas de monitoramento e observabilidade.

Eventos importantes:

- aumento da taxa de erros 5xx;
- falhas repetidas de autenticação;
- erros recorrentes de validação;
- indisponibilidade de serviços externos;
- exceções críticas de infraestrutura.

Esses eventos poderão gerar alertas automáticos para a equipe responsável.

6.17 Internacionalização

As mensagens retornadas ao usuário poderão ser internacionalizadas.

A API poderá utilizar o cabeçalho:

```
Accept-Language: pt-BR
```

Futuras versões poderão suportar outros idiomas mantendo o mesmo contrato de resposta.

6.18 Boas Práticas

Durante a implementação deverão ser observadas as seguintes recomendações:

- utilizar o código HTTP mais adequado para cada situação;
- padronizar todas as respostas de erro;
- nunca expor stack traces ou detalhes internos ao cliente;
- registrar exceções em logs estruturados;
- utilizar `traceld` para correlação;
- documentar todos os erros possíveis no OpenAPI/Swagger;
- diferenciar claramente erros técnicos de erros de negócio.

6.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura em que o tratamento de erros está inserido.
Documento 06 – Arquitetura Backend	Especifica a implementação do tratamento global de exceções e das classes de erro.
Documento 07 – Arquitetura Frontend	Define como a interface interpreta e apresenta os erros retornados pela API.
Documento 08 – Segurança	Define o tratamento dos erros de autenticação, autorização e auditoria.
Documento 09 – Docker	Estabelece as diretrizes de monitoramento e observabilidade utilizadas na correlação dos erros.

620 Conclusão do Capítulo

A padronização dos códigos HTTP, do tratamento global de exceções e das respostas de erro proporciona uma comunicação clara, consistente e previsível entre a API e seus consumidores. A adoção de uma estrutura inspirada na RFC 7807, aliada ao uso de `traceld`, logs estruturados e integração com observabilidade, facilita o diagnóstico de problemas, melhora a experiência dos desenvolvedores e fortalece a confiabilidade da plataforma.

7 – Paginação, Ordenação, Filtros, Pesquisa e Metadados

7.1 Objetivo

Este capítulo estabelece o padrão para operações de consulta que retornam coleções de recursos na API do GTP.

As diretrizes aqui definidas têm como objetivos:

- evitar o retorno de grandes volumes de dados em uma única requisição;
- melhorar o desempenho da aplicação;
- padronizar consultas entre todos os módulos;
- facilitar o consumo da API pelo frontend e por sistemas externos;
- permitir futuras otimizações sem alteração do contrato da API.

7.2 Princípios Gerais

Todas as listagens da API deverão seguir os seguintes princípios:

- paginação obrigatória para coleções;
- ordenação configurável;
- filtros opcionais;
- pesquisa textual quando aplicável;
- retorno de metadados da consulta;
- desempenho previsível;
- compatibilidade entre versões.

7.3 Paginação

A paginação será utilizada para limitar a quantidade de registros retornados em cada resposta.

Parâmetros padrão:

Parâmetro	Tipo	Obrigatório	Descrição
-----------	------	-------------	-----------

page	Integer	Não	Número da página (iniciando em 0).
------	---------	-----	------------------------------------

size	Integer	Não	Quantidade de registros por página.
------	---------	-----	-------------------------------------

Valores padrão recomendados:

```
page = 0
```

```
size = 20
```

Valor máximo recomendado:

```
size = 100
```

Caso o cliente solicite um valor superior ao permitido, a API poderá limitar automaticamente ao máximo configurado.

7.4 Exemplo de Paginação

Consulta:

```
GET /api/v1/publicadores?page=0&size=20
```

Consulta da segunda página:

```
GET /api/v1/publicadores?page=1&size=20
```

Consulta da quinta página:

```
GET /api/v1/publicadores?page=4&size=20
```

7.5 Estrutura da Resposta Paginada

Todas as respostas paginadas seguirão um padrão único.

Exemplo:

```
{
  "success": true,
  "data": [
    {}
  ],
  "page": {
    "number": 0,
    "size": 20,
    "totalElements": 358,
    "totalPages": 18,
    "first": true,
    "last": false,
    "numberOfElements": 20
  }
}
```

7.6 Campos de Paginação

Campo	Descrição
number	Página atual.
size	Quantidade de registros por página.
totalElements	Total de registros encontrados.
totalPages	Número total de páginas.
first	Indica se é a primeira página.
last	Indica se é a última página.
numberOfElements	Quantidade efetivamente retornada.

7.7 Ordenação

A API permitirá ordenar resultados por um ou mais campos.

Parâmetros:

Parâmetro Finalidade

sort Campo utilizado na ordenação.

direction ASC ou DESC.

Exemplo:

```
GET
/api/v1/publicadores?sort=nomeCompleto&direction=ASC
```

Outro exemplo:

```
GET
/api/v1/publicadores?sort=dataCadastro&direction=DESC
```

7.8 Ordenação por Múltiplos Campos

Quando necessário, será possível ordenar por mais de um atributo.

Exemplo:

```
GET /api/v1/publicadores?sort=congregacao, nomeCompleto
```

Ou utilizando múltiplos parâmetros:

```
GET
/api/v1/publicadores?sort=congregacao,ASC&sort=nomeCompleto,ASC
```

A implementação deverá manter comportamento consistente para todas as coleções.

7.9 Pesquisa Textual

Recursos que suportarem pesquisa deverão utilizar o parâmetro:

```
search
```

Exemplo:

```
GET /api/v1/publicadores?search=João
```

A pesquisa poderá abranger:

- nome completo;
- telefone;
- e-mail;

- código do publicador.

Os campos efetivamente pesquisados deverão ser documentados para cada recurso.

7.10 Filtros Simples

Os filtros deverão utilizar parâmetros de consulta.

Exemplos:

```
GET /api/v1/publicadores?ativo=true
```

```
GET /api/v1/publicadores?sexo=MASCULINO
```

```
GET /api/v1/publicadores?cargo=ANCIAO
```

Cada filtro será opcional.

7.11 Filtros Compostos

Filtros poderão ser combinados.

Exemplo:

```
GET  
/api/v1/publicadores?ativo=true&cargo=ANCIAO&congregacaoId=12
```

Outro exemplo:

```
GET  
/api/v1/territorios?status=DISPONIVEL&bairro=Centro
```

A combinação dos filtros deverá resultar na interseção dos critérios informados.

7.12 Filtros por Intervalo

Alguns recursos permitirão filtros por intervalo.

Exemplo:

```
GET /api/v1/designacoes?dataInicial=2026-01-01&dataFinal=2026-12-31
```

Outros exemplos:

- intervalo de datas;
- intervalo de valores;
- período de auditoria.

7.13 Filtros Booleanos

Campos booleanos utilizarão os valores:

```
true  
false
```

Exemplo:

```
GET /api/v1/publicadores?ativo=true
```

7.14 Filtros por Enumeração

Campos baseados em enumerações utilizarão seus valores oficiais.

Exemplo:

```
GET /api/v1/territorios?status=EM_USO
```

Outro exemplo:

```
GET /api/v1/designacoes?tipo=REVISITA
```

Os valores aceitos deverão ser documentados para cada endpoint.

7.15 Metadados

Além dos dados da consulta, a resposta poderá conter metadados.

Exemplo:

```
{  
  "meta": {  
    "apiVersion": "v1",  
    "processingTime": 28,  
    "generatedAt": "2026-07-10T18:45:00Z"  
  }  
}
```

Esses metadados auxiliam monitoramento, diagnóstico e auditoria.

7.16 Desempenho das Consultas

Para garantir eficiência, deverão ser adotadas as seguintes práticas:

- paginação obrigatória;
- utilização de índices no banco de dados;

- consultas otimizadas;
- projeções quando aplicável;
- carregamento controlado de relacionamentos;
- limitação do tamanho máximo das páginas.

7.17 Respostas Vazias

Quando nenhum registro atender aos critérios informados, a API deverá retornar:

```
{
  "success": true,
  "data": [],
  "page": {
    "number": 0,
    "size": 20,
    "totalElements": 0,
    "totalPages": 0,
    "first": true,
    "last": true,
    "numberOfElements": 0
  }
}
```

A ausência de resultados não deverá ser tratada como erro.

7.18 Consultas Complexas

Filtros muito específicos ou combinações avançadas poderão utilizar objetos de pesquisa enviados no corpo da requisição, quando tecnicamente justificável.

Exemplo:

POST /api/v1/publicadores/search

Essa abordagem deverá ser utilizada apenas quando os parâmetros de consulta não forem suficientes para representar a pesquisa de forma clara.

7.19 Boas Práticas

Durante a implementação das consultas deverão ser observadas as seguintes recomendações:

- limitar o tamanho máximo das páginas;
- utilizar índices para campos frequentemente pesquisados;
- evitar consultas que retornem todos os registros sem paginação;
- documentar todos os filtros disponíveis;
- manter nomenclatura consistente para parâmetros;
- garantir ordenação determinística;
- utilizar projeções para reduzir tráfego quando apropriado.

7.20 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral que suporta as operações de consulta.
Documento 06 – Arquitetura Backend	Implementa paginação, filtros e ordenação utilizando Spring Data JPA.
Documento 07 – Arquitetura Frontend	Consome as estruturas paginadas e apresenta os controles de navegação.
Documento 08 – Segurança	Define restrições de acesso que podem influenciar os resultados retornados.
Documento 09 – Docker	Estabelece a infraestrutura de monitoramento utilizada para avaliar o desempenho das consultas.

7.21 Conclusão do Capítulo

A padronização da paginação, ordenação, filtros, pesquisa e metadados garante que todas as consultas da API do GTP apresentem comportamento uniforme, previsível e eficiente. Essas diretrizes favorecem o desempenho da aplicação, simplificam a integração com o frontend e proporcionam uma base sólida para a evolução da plataforma, mesmo diante de grandes volumes de dados.

8 – Documentação OpenAPI

8.1 Objetivo

Este capítulo estabelece as diretrizes para documentação da API REST do GTP utilizando a especificação **OpenAPI 3.1**.

Os principais objetivos são:

- documentar todos os endpoints da API;
- manter a documentação sincronizada com a implementação;
- facilitar o consumo da API por desenvolvedores;
- apoiar testes, integrações e geração de clientes;
- fornecer exemplos claros de requisições e respostas.

A documentação deverá ser considerada parte integrante do processo de desenvolvimento.

8.2 OpenAPI

O GTP adotará o padrão **OpenAPI 3.1** como especificação oficial da API.

A especificação permitirá descrever:

- recursos disponíveis;
- operações suportadas;
- parâmetros;
- modelos de dados;
- códigos HTTP;
- autenticação;
- exemplos de uso;
- mensagens de erro.

A documentação será gerada automaticamente a partir do código do backend, reduzindo divergências entre documentação e implementação.

8.3 Benefícios

A adoção do OpenAPI proporciona diversas vantagens.

Padronização

Todos os endpoints seguem um formato uniforme.

Documentação Automática

A documentação é produzida diretamente a partir das anotações do código.

Geração de Clientes

Ferramentas podem gerar SDKs para diferentes linguagens.

Testes

É possível executar chamadas diretamente pela interface de documentação.

Evolução Controlada

Mudanças na API podem ser acompanhadas por meio do versionamento da especificação.

8.4 Swagger UI

O Swagger UI será utilizado como interface web interativa para exploração da API.

Principais funcionalidades:

- visualização dos endpoints;
- execução de requisições;
- autenticação via JWT;
- visualização dos modelos JSON;
- consulta dos códigos HTTP;
- inspeção de exemplos de respostas.

Durante o desenvolvimento, essa interface será uma das principais ferramentas de validação da API.

8.5 Estrutura da Documentação

A documentação deverá estar organizada em grupos funcionais.

Exemplo:

Grupo	Endpoints
Autenticação	Login, Logout, Refresh Token
Usuários	CRUD de usuários
Congregações	Gestão de congregações
Territórios	Cadastro e gerenciamento de territórios

Grupo	Endpoints
Quadras	Gestão de quadras
Endereços	Endereços vinculados aos territórios
Publicadores	Cadastro e gerenciamento
Designações	Controle de designações
Visitas	Registro de visitas
Estudos Bíblicos	Gestão dos estudos
Relatórios	Emissão de relatórios
Dashboard	Indicadores e estatísticas
Configurações	Parametrizações do sistema
Auditoria	Consulta de eventos registrados

Essa organização facilita a navegação e a localização dos recursos.

8.6 Informações Gerais da API

A documentação deverá apresentar informações institucionais.

Exemplo:

Campo	Conteúdo
--------------	-----------------

Nome	API REST - Gestor de Territórios e Publicações
------	--

Versão	v1
--------	----

Descrição	API oficial do sistema GTP
-----------	----------------------------

Licença	Conforme política do projeto
---------	------------------------------

Contato	Equipe de desenvolvimento
---------	---------------------------

Ambiente Desenvolvimento, Homologação e Produção

Esses dados auxiliam consumidores e integradores.

8.7 Documentação dos Endpoints

Cada endpoint deverá conter, no mínimo:

- resumo da operação;

- descrição detalhada;
- método HTTP;
- URL;
- parâmetros;
- corpo da requisição;
- respostas possíveis;
- códigos HTTP;
- requisitos de autenticação;
- papéis autorizados.

Essa documentação deverá ser mantida sempre atualizada.

8.8 Documentação dos Parâmetros

Todos os parâmetros deverão ser descritos.

Exemplo:

Campo	Descrição
nome	Nome do parâmetro.
Tipo	Tipo de dado.
Obrigatório	Sim ou Não.
Descrição	Finalidade do parâmetro.
Exemplo	Valor de exemplo.

Essa padronização reduz dúvidas durante o consumo da API.

8.9 Documentação dos Modelos

Todos os DTOs utilizados pela API deverão ser documentados.

Cada modelo deverá apresentar:

- nome;
- descrição;
- propriedades;
- tipo de cada propriedade;
- obrigatoriedade;

- exemplo de preenchimento.

Exemplo simplificado:

```
{
  "nomeCompleto": "Carlos Alberto Souza",
  "email": "carlos@email.com",
  "telefone": "(55)99999-1111",
  "ativo": true
}
```

8.10 Exemplos de Requisições

Sempre que possível, a documentação deverá apresentar exemplos completos de requisições.

Exemplo:

```
POST /api/v1/publicadores
{
  "nomeCompleto": "Carlos Alberto Souza",
  "telefone": "(55)99999-1111",
  "email": "carlos@email.com"
}
```

Esses exemplos facilitam testes e integração.

8.11 Exemplos de Respostas

As respostas também deverão conter exemplos.

Exemplo:

```
{
  "success": true,
  "data": {
    "id": 125,
    "nomeCompleto": "Carlos Alberto Souza"
  }
}
```

8.12 Documentação dos Erros

Todos os possíveis erros deverão estar documentados.

Exemplo:

Código HTTP Situação

400	Dados inválidos.
401	Não autenticado.
403	Acesso negado.
404	Recurso não encontrado.
409	Conflito de dados.
422	Violação de regra de negócio.
500	Erro interno do servidor.

Cada erro deverá apresentar descrição e exemplo de resposta.

8.13 Segurança na Documentação

Os mecanismos de autenticação deverão estar documentados.

Itens obrigatórios:

- autenticação JWT;
- formato do cabeçalho Authorization;
- renovação de tokens;
- recursos públicos;
- recursos protegidos.

A documentação deverá permitir que o desenvolvedor informe um token JWT para testar endpoints protegidos.

8.14 Versionamento da Especificação

A especificação OpenAPI acompanhará o versionamento da API.

Exemplo:

API Especificação

v1 OpenAPI v1

API Especificação

v2 OpenAPI v2

Mudanças incompatíveis deverão resultar em nova versão da especificação.

8.15 Exportação

A especificação OpenAPI poderá ser exportada em formatos amplamente utilizados.

Exemplos:

- JSON;
- YAML.

Esses formatos permitem integração com ferramentas de geração de código, testes automatizados e plataformas de gerenciamento de APIs.

8.16 Integração com Ferramentas

A documentação deverá ser compatível com ferramentas como:

- Swagger UI;
- OpenAPI Generator;
- Postman;
- Insomnia;
- ferramentas de testes automatizados;
- plataformas de integração contínua.

Essa compatibilidade reduz o esforço de integração entre equipes.

8.17 Atualização da Documentação

A documentação deverá ser atualizada sempre que houver:

- inclusão de novos endpoints;
- alteração de parâmetros;
- mudança em modelos de dados;
- novos códigos de resposta;
- evolução da segurança;
- alterações de regras de negócio refletidas na API.

Alterações no código e na documentação deverão ocorrer no mesmo ciclo de desenvolvimento.

8.18 Boas Práticas

Durante a elaboração da documentação deverão ser observadas as seguintes recomendações:

- documentar todos os endpoints públicos;
- manter exemplos atualizados;
- utilizar descrições claras e objetivas;
- documentar parâmetros obrigatórios e opcionais;
- informar todos os códigos HTTP possíveis;
- documentar requisitos de autenticação e autorização;
- evitar informações redundantes;
- revisar periodicamente a consistência entre documentação e implementação.

8.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral da solução.
Documento 06 – Arquitetura Backend	Implementa os endpoints e as anotações utilizadas para gerar a documentação OpenAPI.
Documento 07 – Arquitetura Frontend	Consome os contratos documentados pela especificação OpenAPI.
Documento 08 – Segurança	Define os mecanismos de autenticação e autorização documentados na API.
Documento 09 – Docker	Disponibiliza a infraestrutura para publicação da API e de sua documentação.

8.20 Conclusão do Capítulo

A adoção do OpenAPI 3.1 como padrão de documentação da API do GTP garante uma documentação consistente, atualizada e integrada ao processo de desenvolvimento. A geração automática da especificação, aliada ao uso do Swagger UI, facilita o

entendimento dos contratos da API, melhora a produtividade das equipes, reduz inconsistências entre código e documentação e simplifica a integração com clientes, ferramentas de teste e plataformas de desenvolvimento.

9 – Endpoints da API REST

9.1 Objetivo

Este capítulo apresenta a especificação funcional completa dos endpoints da API REST do **Gestor de Territórios e Publicações (GTP)**.

Cada endpoint representa um contrato oficial entre os consumidores da API e o backend, definindo:

- URI;
- método HTTP;
- autenticação necessária;
- papéis autorizados;
- parâmetros;
- corpo da requisição;
- estrutura da resposta;
- códigos HTTP;
- regras de negócio;
- observações técnicas.

Todos os endpoints descritos neste documento deverão ser implementados conforme os contratos aqui definidos.

9.2 Organização dos Endpoints

Os endpoints serão organizados por módulos de negócio.

Módulo	Prefixo
Autenticação	/api/v1/auth
Usuários	/api/v1/usuarios
Congregações	/api/v1/congregacoes
Territórios	/api/v1/territorios
Quadras	/api/v1/quadras

Módulo	Prefixo
Endereços	/api/v1/enderecos
Publicadores	/api/v1/publicadores
Designações	/api/v1/designacoes
Visitas	/api/v1/visitas
Estudos Bíblicos	/api/v1/estudos-biblicos
Publicações	/api/v1/publicacoes
Relatórios	/api/v1/relatorios
Dashboard	/api/v1/dashboard
Configurações	/api/v1/configuracoes
Auditoria	/api/v1/auditoria

9.3 Padrão de Especificação dos Endpoints

Todos os endpoints seguirão o mesmo formato de documentação.

Cada operação apresentará:

- objetivo;
- URL;
- método HTTP;
- autenticação;
- perfis autorizados;
- parâmetros;
- corpo da requisição;
- resposta de sucesso;
- respostas de erro;
- regras de negócio;
- observações.

Esse padrão garante uniformidade em toda a documentação.

9.4 Módulo de Autenticação

Base URL:

`/api/v1/auth`

9.4.1 Login

Objetivo

Autenticar um usuário e emitir os tokens de acesso.

Endpoint

`POST /api/v1/auth/login`

Autenticação

Não requerida.

Request

```
{
  "username": "usuario@email.com",
  "password": "senha"
}
```

Response (200 OK)

```
{
  "accessToken": "jwt-token",
  "refreshToken": "refresh-token",
  "tokenType": "Bearer",
  "expiresIn": 1800
}
```

Possíveis respostas

Código Situação

- | | |
|-----|------------------------------|
| 200 | Login realizado com sucesso. |
| 400 | Dados inválidos. |
| 401 | Credenciais incorretas. |
| 423 | Usuário bloqueado. |

9.4.2 Renovação de Token

`POST /api/v1/auth/refresh`

Objetivo:

Emitir novo Access Token.

Resposta:

```
{
  "accessToken": "...",
  "expiresIn": 1800
}
```

9.4.3 Logout

`POST /api/v1/auth/logout`

Objetivo:

Invalidar Refresh Token.

9.5 Módulo de Usuários

Base URL

`/api/v1/usuarios`

9.5.1 Listar Usuários

`GET /api/v1/usuarios`

Permissões:

- ADMINISTRADOR

Parâmetros:

- page
- size
- sort
- search

Resposta:

```
{
  "data": []
}
```

9.5.2 Buscar Usuário

GET /api/v1/usuarios/{id}

9.5.3 Criar Usuário

POST /api/v1/usuarios

9.5.4 Atualizar Usuário

PUT /api/v1/usuarios/{id}

9.5.5 Atualização Parcial

PATCH /api/v1/usuarios/{id}

9.5.6 Excluir Usuário

DELETE /api/v1/usuarios/{id}

9.6 Módulo de Congregações

Endpoints principais:

GET /congregacoes
GET /congregacoes/{id}
POST /congregacoes
PUT /congregacoes/{id}
PATCH /congregacoes/{id}
DELETE /congregacoes/{id}

Recursos adicionais:

GET /congregacoes/{id}/publicadores
GET /congregacoes/{id}/territorios
GET /congregacoes/{id}/estatisticas

9.7 Módulo de Territórios

Principais endpoints

GET /territorios

```
GET /territorios/{id}
POST /territorios
PUT /territorios/{id}
DELETE /territorios/{id}
```

Operações específicas

```
POST /territorios/{id}/designar
POST /territorios/{id}/devolver
POST /territorios/{id}/concluir
GET /territorios/disponiveis
GET /territorios/em-uso
GET /territorios/atrasados
```

9.8 Módulo de Quadras

```
GET /quadras
GET /quadras/{id}
POST /quadras
PUT /quadras/{id}
DELETE /quadras/{id}
```

9.9 Módulo de Endereços

```
GET /enderecos
POST /enderecos
PUT /enderecos/{id}
DELETE /enderecos/{id}
```

9.10 Módulo de Publicadores

Principais operações

```
GET /publicadores
GET /publicadores/{id}
POST /publicadores
PUT /publicadores/{id}
PATCH /publicadores/{id}
```

DELETE /publicadores/{id}

Operações específicas

GET /publicadores/aniversariantes

GET /publicadores/ativos

GET /publicadores/inativos

GET /publicadores/{id}/territorios

GET /publicadores/{id}/designacoes

9.11 Módulo de Designações

GET /designacoes

POST /designacoes

PUT /designacoes/{id}

DELETE /designacoes/{id}

Operações

POST /designacoes/{id}/aceitar

POST /designacoes/{id}/cancelar

POST /designacoes/{id}/concluir

9.12 Módulo de Visitas

GET /visitas

POST /visitas

PUT /visitas/{id}

DELETE /visitas/{id}

9.13 Módulo de Estudos Bíblicos

GET /estudos-biblicos

POST /estudos-biblicos

PUT /estudos-biblicos/{id}

DELETE /estudos-biblicos/{id}

9.14 Módulo de Publicações

GET /publicacoes

POST /publicacoes

PUT /publicacoes/{id}

DELETE /publicacoes/{id}

9.15 Módulo Dashboard

GET /dashboard

Indicadores

GET /dashboard/resumo

GET /dashboard/graficos

GET /dashboard/estatisticas

9.16 Módulo Relatórios

GET /relatorios

Exportações

GET /relatorios/pdf

GET /relatorios/excel

GET /relatorios/csv

Relatórios disponíveis

- territórios
- publicadores
- designações
- visitas
- auditoria
- estudos bíblicos
- estatísticas gerais

9.17 Módulo Configurações

GET /configuracoes

PUT /configuracoes

9.18 Módulo Auditoria

GET /auditoria

Filtros

GET /auditoria?usuario=15

GET /auditoria?acao=LOGIN

GET /auditoria?dataInicial=...

GET /auditoria?dataFinal=...

9.19 Versionamento dos Endpoints

Todos os recursos seguirão:

/api/v1/

Mudanças incompatíveis deverão originar uma nova versão da API.

9.20 Convenções Gerais

Todos os endpoints deverão:

- utilizar HTTPS;
- retornar JSON;
- exigir autenticação quando necessário;
- documentar exemplos no OpenAPI;
- utilizar DTOs;
- retornar códigos HTTP padronizados;
- registrar auditoria quando aplicável;
- respeitar as regras de autorização definidas no Documento 08.

9.21 Relação com os Demais Documentos

Documento	Relação
Documento 03 – Regras de Negócio	Define as regras aplicadas pelos endpoints.
Documento 04 – Casos de Uso	Origina as operações expostas pela API.

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral em que a API está inserida.
Documento 06 – Arquitetura Backend	Implementa os Controllers, Services e Repositories correspondentes aos endpoints.
Documento 07 – Arquitetura Frontend	Consome todos os recursos definidos neste capítulo.
Documento 08 – Segurança	Define autenticação, autorização e auditoria dos endpoints.

9.22 Conclusão do Capítulo

Este capítulo apresentou a estrutura funcional da API REST do GTP, organizando os endpoints por domínio de negócio e estabelecendo um padrão uniforme para sua documentação. Os contratos aqui definidos servirão como base para a implementação do backend, o consumo pelo frontend, a geração automática da documentação OpenAPI e a criação de testes automatizados, garantindo consistência, segurança e evolução controlada da plataforma.

10 – Versionamento, Evolução da API, Boas Práticas e Governança

10.1 Objetivo

Este capítulo estabelece as políticas de versionamento, evolução, governança e manutenção da API REST do GTP.

Os objetivos são:

- garantir compatibilidade entre versões;
- permitir evolução contínua da API;
- reduzir impactos para consumidores existentes;
- estabelecer padrões de governança técnica;
- definir boas práticas para desenvolvimento e manutenção.

Essas diretrizes deverão ser observadas durante todo o ciclo de vida da API.

10.2 Estratégia de Versionamento

A API utilizará versionamento por URI.

Exemplo:

```
/api/v1/
```

Versões futuras:

```
/api/v2/
```

```
/api/v3/
```

Essa abordagem facilita a coexistência de múltiplas versões e reduz o impacto de alterações incompatíveis.

10.3 Tipos de Alterações

As alterações na API serão classificadas em:

Alterações Compatíveis

Não exigem mudança de versão principal.

Exemplos:

- inclusão de novos endpoints;
- adição de campos opcionais;
- melhoria de desempenho;
- correções de defeitos;
- inclusão de novos códigos de enumeração compatíveis.

Alterações Incompatíveis

Exigem nova versão principal.

Exemplos:

- remoção de endpoints;
- alteração de contratos obrigatórios;
- mudança de estrutura das respostas;
- alteração de semântica de operações;
- remoção de campos públicos.

10.4 Compatibilidade Retroativa

Sempre que possível, a API deverá manter compatibilidade com versões anteriores (*backward compatibility*).

Princípios:

- preservar contratos existentes;
- evitar mudanças abruptas;
- manter comportamento previsível;
- documentar alterações relevantes.

10.5 Depreciação de Endpoints

Antes da remoção de qualquer endpoint, deverá existir um período formal de depreciação.

O processo deverá incluir:

1. identificação do endpoint como obsoleto;
2. documentação da alternativa recomendada;
3. comunicação aos consumidores;
4. manutenção durante período de transição;
5. remoção apenas na próxima versão principal.

10.6 Ciclo de Vida dos Endpoints

Cada endpoint seguirá um ciclo de vida definido.

Estado	Descrição
Planejado	Definido na documentação, ainda não implementado.
Em Desenvolvimento	Implementação em andamento.
Disponível	Endpoint publicado e suportado.
Depreciado	Uso desencorajado, substituição disponível.
Descontinuado	Não disponível em novas versões.

Essa classificação facilita o gerenciamento da evolução da API.

10.7 Governança da API

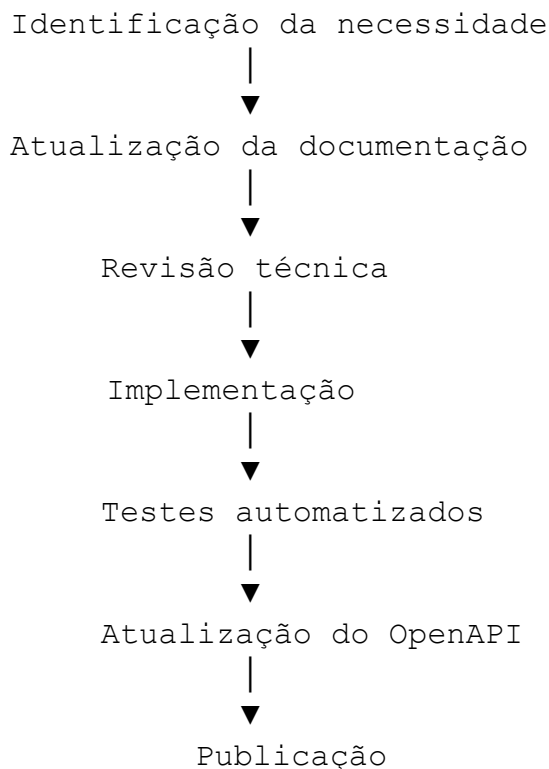
A governança da API será baseada nos seguintes princípios:

- contratos documentados;
- revisão técnica obrigatória;
- padronização de nomenclatura;
- reutilização de componentes;
- monitoramento contínuo;
- controle de mudanças;
- documentação sincronizada com a implementação.

Toda alteração deverá ser registrada e aprovada conforme o processo de desenvolvimento adotado pelo projeto.

10.8 Processo de Evolução

A inclusão de novos recursos deverá seguir o seguinte fluxo:



Esse processo garante rastreabilidade e qualidade nas evoluções da API.

10.9 Boas Práticas de Desenvolvimento

Todos os novos endpoints deverão:

- seguir os padrões REST definidos neste documento;
- utilizar DTOs para entrada e saída;
- documentar parâmetros e exemplos;
- utilizar códigos HTTP apropriados;
- respeitar as políticas de autenticação e autorização;
- registrar eventos relevantes para auditoria;
- possuir testes automatizados.

10.10 Qualidade da API

A qualidade da API será monitorada por meio de indicadores como:

Indicador	Objetivo
Disponibilidade	Garantir alta disponibilidade do serviço.
Tempo médio de resposta	Monitorar desempenho.
Taxa de erros 4xx	Identificar problemas de consumo.
Taxa de erros 5xx	Identificar falhas internas.
Cobertura de testes	Garantir confiabilidade.
Tempo de implantação	Avaliar eficiência do processo de entrega.

Esses indicadores deverão ser acompanhados periodicamente.

10.11 Monitoramento

A API deverá fornecer informações para monitoramento operacional.

Itens monitorados:

- quantidade de requisições;
- tempo médio de resposta;
- utilização dos endpoints;
- consumo por cliente;
- falhas de autenticação;

- exceções;
- utilização de recursos críticos.

Esses dados apoiarão a melhoria contínua da plataforma.

10.12 Observabilidade

A observabilidade será baseada em três pilares:

- **Logs:** registro estruturado de eventos e exceções.
- **Métricas:** indicadores de desempenho e utilização.
- **Rastreamento (Tracing):** correlação de requisições por meio de identificadores únicos.

Essa abordagem facilita a identificação de problemas em ambientes distribuídos.

10.13 Testes

A evolução da API deverá ser acompanhada por testes automatizados.

Tipos de testes recomendados:

- testes unitários;
- testes de integração;
- testes de contrato;
- testes de regressão;
- testes de segurança;
- testes de desempenho.

Novos endpoints somente deverão ser publicados após aprovação da suíte de testes.

10.14 Documentação Contínua

A documentação da API deverá evoluir juntamente com o código-fonte.

Sempre que houver:

- inclusão de endpoints;
- alteração de contratos;
- mudanças em parâmetros;
- novos modelos de dados;
- atualização das regras de negócio;

a documentação OpenAPI e este documento deverão ser atualizados na mesma entrega.

10.15 Integração Contínua

A API deverá integrar-se ao processo de CI/CD do projeto.

O pipeline deverá contemplar:

1. compilação do projeto;
2. execução de testes automatizados;
3. análise estática de código;
4. geração da documentação OpenAPI;
5. empacotamento da aplicação;
6. criação da imagem Docker;
7. implantação nos ambientes correspondentes.

Essa automação reduz erros e aumenta a confiabilidade das entregas.

10.16 Segurança na Evolução

Toda alteração deverá preservar as políticas de segurança estabelecidas.

Antes da publicação de uma nova versão, deverão ser verificados:

- autenticação;
- autorização;
- validação de entradas;
- proteção contra vulnerabilidades conhecidas;
- conformidade com as diretrizes do Documento 08 – Segurança.

10.17 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral que suporta a evolução da API.
Documento 06 – Arquitetura Backend	Implementa os componentes descritos neste documento.

Documento	Relação
Documento 07 – Arquitetura Frontend	Consome e adapta-se às versões da API.
Documento 08 – Segurança	Define as políticas de segurança aplicáveis às evoluções da API.
Documento 09 – Docker	Suporta os processos de build, publicação e implantação.

10.18 Consolidação do Documento

O **Documento 10 – API REST** estabelece o contrato oficial de comunicação do GTP entre clientes e servidor.

Ao longo deste documento foram definidos:

- princípios REST;
- arquitetura geral da API;
- padrões de recursos e endpoints;
- autenticação e autorização;
- estrutura de requisições e respostas;
- tratamento de erros;
- paginação, filtros e ordenação;
- documentação OpenAPI;
- especificação dos endpoints;
- versionamento e governança.

Essas definições garantem uma API consistente, segura, escalável e preparada para evolução contínua.

Conclusão Geral

A API REST do Gestor de Territórios e Publicações foi projetada para atender aos princípios de interoperabilidade, baixo acoplamento e alta coesão, utilizando tecnologias modernas do ecossistema Java e padrões amplamente adotados pelo mercado.

A combinação de Spring Boot, Spring Security, JWT, OpenAPI, DTOs, tratamento centralizado de exceções, versionamento controlado e documentação automatizada

fornece uma base sólida para o desenvolvimento do backend e para a integração com o frontend React, aplicações móveis e sistemas externos.

Este documento passa a ser a referência oficial para implementação, testes, integração, manutenção e evolução da API do GTP, assegurando consistência entre as equipes de desenvolvimento e contribuindo para a qualidade e a sustentabilidade da plataforma.